

Тема урока: полиморфизм

1. Понятие полиморфизма
2. Полиморфизм операторов
3. Полиморфизм функций
4. Полиморфизм в методах классов

Аннотация. Урок посвящен полиморфизму в Python.

Понятие полиморфизма

Полиморфизм — это множество форм. Однако в понятиях ООП имеется в виду, скорее, обратное. Объекты разных классов, с разной внутренней реализацией, могут иметь одинаковые интерфейсы. Например, для чисел есть операция сложения, обозначаемая знаком `+`. Однако мы можем определить класс, объекты которого также будут поддерживать операцию, обозначаемую этим знаком, но это вовсе не значит, что объекты должны быть числами и будет получаться какая-то сумма. Операция `+` для объектов нашего класса может значить что-то иное. Но интерфейс, в данном случае это знак `+`, у чисел и нашего класса будет одинаков. Полиморфность же проявляется во внутренней реализации и результате операции.

Полиморфизм операторов

Мы уже сталкивались с полиморфизмом операции `+` в языке Python. Для чисел она обозначает сложение, а для строк — конкатенацию. Внутренняя реализация оператора `+` существенно отличается для целых чисел, чисел с плавающей точкой и строк. То есть на самом деле это три разные операции — интерпретатор Python выбирает одну из них при выполнении в зависимости от операндов.

Приведенный ниже код:

```
print(10 + 20)
print(10.5 + 20.8)
print('bee' + 'geek')
```

ВЫВОДИТ:

30
31.3
beegeek

Здесь мы можем увидеть единственный оператор `+`, выполняющий разные операции для разных типов данных. Это один из самых простых примеров полиморфизма в Python, благодаря которому мы можем выполнять разные действия одинаковыми способами, другими словами, определять один интерфейс и иметь множество реализаций.

Полиморфизм функций

В Python есть некоторые функции, которые могут работать с объектами разных типов. Одной из таких функций является функция `len()`.

Приведенный ниже код:

```
print(len('beegeek'))  
print(len(['beegeek', 'stepik', 'python']))  
print(len({'name': 'Timur', 'city': 'Dubai'}))
```

ВЫВОДИТ:

7
3
2

Здесь мы можем увидеть, что различные типы данных, такие как строка, список, кортеж, множество и словарь могут быть обработаны функцией `len()`. Для каждого типа данных она возвращает специфичную для него информацию: для строк — количество символов, для списка — количество элементов, для словаря — количество пар ключ-значение.

Полиморфизм в методах классов

В Python различные классы могут иметь методы с одинаковыми именами, что позволяет нам при их определении использовать концепцию полиморфизма, с помощью которого мы сможем обобщить вызов этих методов и игнорировать тип объекта, с которым работаем.

Рассмотрим похожие по структуре классы `Cat` и `Dog`, описывающие кошку и собаку соответственно:

```
class Cat:
    def __init__(self, name):
        self.name = name

    def sound(self):
        return 'мяу'

class Dog:
    def __init__(self, name):
        self.name = name

    def sound(self):
        return 'гав'
```

Экземпляры обоих классов имеют атрибут `name` и метод `sound()`, и концепция полиморфизма в данном случае заключается в том, что нам неважно, с экземпляром класса `Cat` или `Dog` мы работаем, главное, что мы знаем о наличии у него атрибута и метода, к которым можем обратиться.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self.name = name

    def sound(self):
        return 'мяу'

class Dog:
    def __init__(self, name):
        self.name = name

    def sound(self):
        return 'гав'

cat = Cat('Кемаль')
dog = Dog('Роджер')

for animal in (cat, dog):
    print(f'{animal.name} говорит {animal.sound()}')
```

ВЫВОДИТ:

```
Кемаль говорит мяу
Роджер говорит гав
```

Обратите внимание, что мы не создавали общего класса-родителя и не соединяли классы вместе каким-либо другим способом. Тем не менее, мы можем объединить экземпляры этих классов в кортеж и проитерироваться по нему, используя общую переменную `animal`. Если переменная `animal` ссылается на экземпляр класса `Cat`, выполняются методы, определенные в этом классе, если на экземпляр класса `Dog` — методы класса `Dog`.

Также тело цикла может быть заменено на вызов полиморфной функции, которая будет верно работать с экземплярами обоих классов, благодаря наличию у них атрибута `name` и метода `sound()`.

Приведенный ниже код:

```
def say(animal):
    return f'{animal.name} говорит {animal.sound()}'

class Cat:
    def __init__(self, name):
        self.name = name

    def sound(self):
        return 'мяу'

class Dog:
    def __init__(self, name):
        self.name = name

    def sound(self):
        return 'гав'

cat = Cat('Кемаль')
dog = Dog('Роджер')

for animal in (cat, dog):
    print(say(animal))
```

ВЫВОДИТ:

```
Кемаль говорит мяу
Роджер говорит гав
```

Данный подход позволяет заранее написать функцию, которая будет работать со всеми экземплярами любых классов — даже еще не существующих. Важно лишь, чтобы эти классы поддерживали необходимый функции интерфейс.

Чтобы полиморфизм работал, за ним надо следить как на уровне синтаксиса (одинаковые имена методов и количество параметров), так и на уровне смысла (методы с одинаковыми именами делают похожие операции, параметры и возвращаемые значения методов имеют тот же смысл). Например, если в одном из классов, представленных выше, один из методов `sound()` будет не возвращать строку, а печатать, то синтаксически все будет верно, однако результат окажется неверным.

Приведенный ниже код:

```
class Cat:
    def __init__(self, name):
        self.name = name

    def sound(self):
        print('мяу')

class Dog:
    def __init__(self, name):
        self.name = name

    def sound(self):
        return 'гав'

cat = Cat('Кемаль')
dog = Dog('Роджер')

for animal in (cat, dog):
    print(f'{animal.name} говорит {animal.sound()}')
```

ВЫВОДИТ:

```
мяу
Кемаль говорит None
Роджер говорит гав
```

Примечания

Примечание 1. Полиморфизм в переводе с греческого означает нечто, что обладает множеством форм. Однако в ООП смысл немного иной — есть одна форма (интерфейс), но разные содержания (реализации).

Примечание 2. Без полиморфизма может потребоваться проверка типа перед выполнением действия над объектом, чтобы определить правильный метод для вызова. Полиморфизм позволяет избавиться от лишних условий и избыточного кода в целом.

Примечание 3. Python является языком с динамической типизацией, поэтому на самом деле с полиморфизмом мы неявно знакомы с самого начала его изучения. Например, обычный оператор `+` для сложения чисел является полиморфным, и может работать с объектами других типов, однако уже выполнять иную операцию от сложения чисел.

Приведенный ниже код:

```
def f(x, y):
    return x + y

print(f(1, 2))
print(f(1.5, 2.5))
print(f([1, 2], [3, 4]))
```

ВЫВОДИТ:

```
3
4.0
[1, 2, 3, 4]
```

В языках со статической типизацией подобное невозможно, так как в них каждая переменная имеет определенный тип. Поэтому для определения функции, которая бы, помимо сложения, скажем, целых чисел, складывала и вещественные числа, для каждого случая нам бы пришлось определить отдельную реализацию.

Функция для сложения целых и вещественных чисел, а также их комбинаций на примере C++:

```
#include <iostream>

using namespace std;

int f(int x, int y )
{
    return x + y;
}
double f(int x, double y)
{
    return x + y;
}
double f(double x, int y)
{
    return x + y;
}
double f(double x, double y)
{
    return x + y;
}
```

Примечание 4. Перегрузка методов — создание методов с одним и тем же именем, но с разными

